Lesson 4 — Consistency, Replication, Fault Tolerance and Security

INTE 22253 | Source: Lecture 04 (45 slides): consistency models, replication, CAP theorem, fault tolerance, failure detection, reliable communication, commit protocols, DFS security | Builds on Lesson 1 (CAP, failure models)

LEARNING OBJECTIVES (SLIDE 4)

- Distinguish **data-centric** from **client-centric** consistency models and give an example of each.
- Explain replica placement strategies and the trade-offs in choosing where copies live.
- State the **CAP theorem** and classify real systems as CP, AP or CA; explain PACELC.
- Describe failure models and how **heartbeat** and **gossip** protocols detect failures.
- Compare reliable client-server communication with reliable group (multicast) communication.
- Walk through **Two-Phase and Three-Phase Commit** and explain why 3PC exists.
- Identify the core access-control mechanisms used to secure distributed file systems.

WHERE WE LEFT OFF (SLIDE 3)

Distributed system: independent computers that appear as one coherent system. **Middleware**: software layer that hides heterogeneity and gives a uniform interface. **Transparency goals**: access, location, migration, replication, concurrency, failure. **Why replicate?** To improve reliability and performance — but every copy raises a new question: *are all copies the same?* Today's question: once data is copied across nodes, how do we keep those copies close enough to "the truth" for applications to trust them?

1. Why consistency matters (slide 6)

A **replica** is a copy of a data item kept on another node. Replication helps performance, but each replica can be updated independently, so copies can **drift apart**.

CONSISTENCY MODEL (DEFINITION)

A **contract between the data store and processes**: it specifies **what values a read is allowed to return** after a sequence of writes.

- **Stronger models**: easier to reason about, but costlier to implement (more coordination, more latency).
- **Weaker models**: scale better and tolerate partitions, at the cost of applications seeing stale or out-of-order data.

Mental model (slide 6): a client writes `x = 7` on Replica A. What does a read on Replica B or C return right now? The consistency model is the answer to that question.

Notation used below (Tanenbaum): `W(x)a` = write value *a* to item *x*; `R(x)a` = a read of *x* that returned *a*. Each process (P1, P2…) has its own horizontal time line; time runs left to right.

2. Data-centric vs client-centric (slide 7)

	Data-centric models	Client-centric models
Point of view	The **data store's**: rules about the ordering of operations across **all** processes on shared data	A **single (mobile) client's**: what **one** process sees across successive accesses, even as it moves between replicas
Examples	Strict, sequential, causal, eventual	Monotonic reads, monotonic writes, read-your-writes, writes-follow-reads
Best for	Many processes concurrently sharing data (a database, distributed shared memory)	Weakly consistent stores with occasional/lazy replica updates (a mobile calendar app)

3. Data-centric consistency models (slides 8–12)

3.1 Strict consistency — strongest

Rule: any read returns the value of the **most recent write**, as if there were a single global clock and a single copy of the data. Tanenbaum & van Steen: "Any read on a data item x returns a value corresponding to the result of the most recent write on x."

It requires **absolute global time**, which real distributed systems cannot have (network delay, clock skew). So it is **rarely achievable**; it is the theoretical upper bound other models are measured against. Picture: P1 writes, P2 reads an instant later on another machine — the read must already see the new value, even though no message could have travelled that fast.

3.2 Sequential consistency — the practical strong model

Rule: the result of any execution is the same as if the operations of all processes were executed in **some sequential order**, and each process's own operations appear in that order **in the order it issued them**. All processes see the **same interleaving**, but that interleaving **does not have to match real (wall-clock) time**. No global clock is needed, so it is implementable (used in some distributed shared-memory systems).

Worked example: is it sequentially consistent?

P1 does `W(x)a`; P2 does `W(x)b` slightly later in real time. P3 and P4 each read x twice.

Case	P3 reads	P4 reads	Sequentially consistent?
A	`R(x)b` then `R(x)a`	`R(x)b` then `R(x)a`	**Yes.** Both agree on the order W(x)b → W(x)a. It differs from real time (a was written first), but sequential consistency does not care about real time. (Strict consistency would be violated.)
B	`R(x)b` then `R(x)a`	`R(x)a` then `R(x)b`	**No.** P3 saw b before a, P4 saw a before b. There is no single interleaving that both agree on.

Test method: try to write ONE list of all writes that explains every process's reads. If one exists (and keeps each process's own order), the history is sequentially consistent.

3.3 Causal consistency — weaker, widely used

Rule: writes that are **causally related** must be seen by every process in the same order; **concurrent** (unrelated) writes may be seen in different orders by different processes.

Causally related = one write *happened-before* the other, i.e. it could have influenced it: the writer had read or written the earlier value first. Example: a forum reply B that references post A must never be seen by anyone before A; but two unrelated posts C and D can appear in any order. It is cheaper than sequential consistency because unrelated operations need no global agreement.

Process	Operations
P1	`W(x)a` (post A)
P2	`R(x)a`, then `W(x)b` (reply B: it read A first ⇒ A → B are causally related)
P3	`R(x)a`, `R(x)b` — allowed (A before B)
P4	`R(x)b`, `R(x)a` — **violates causal consistency** (sees the reply before the post)

If P2 had **not** read A before writing B, A and B would be concurrent and P4's order would be allowed.

3.4 Eventual consistency — weakest common model

Rule: if no new updates are made, all replicas will **eventually converge** to the same value — no guarantee about ordering or timing in between. "In the absence of new updates, all accesses will eventually return the last updated value." Optimised for **availability and performance**; used in **DNS**, web caches and many NoSQL key-value stores (DynamoDB-style). Trade-off: readers can see **stale data** for an unbounded (but usually short) period. Slide example: R1 has 7; R2 and R3 still show 5 and change to 7 at different moments.

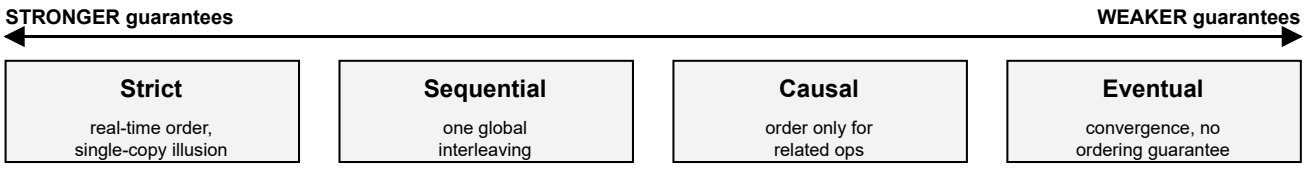


Figure 1 — The consistency spectrum (slide 12). Performance and availability generally increase as guarantees weaken; choosing a model means choosing a point on this line.

4. Client-centric consistency (slides 13–17)

Setting: a mobile client reads and writes through **different replicas** over time (home Wi-Fi replica at t1, office at t2, mobile-data at t3). **Goal:** from that one client's view the world still makes sense, even though replicas sync lazily in the background. The four guarantees are **not alternatives**: a system can, and often does, offer several at once.

Guarantee	Rule (same process)	Ordering	Slide example of what it prevents
Monotonic reads	If a process reads value v of x, any later read by that process never returns an **older** value than v. You never "go backward in time".	read → read	You see 5 new emails on your laptop; on your phone (a lagging replica) you must not see fewer than those 5.
Monotonic writes	A write by a process on x completes before any later write on x by the same process; your writes are applied **everywhere in the order you issued them**.	write → write	Update a document's title, then its body: every replica must apply the title update first. Otherwise a late sync could overwrite a newer value with an older one.
Read your writes	The effect of a write by a process on x is **always seen** by a later read of x by the same process.	write → read	You post a comment, refresh, and it is missing because the read hit a replica without your write. Or: change your password, then a stale replica asks for the old one.
Writes follow reads	A write by a process on x, after it read x, takes place on the **same or a more recent** value than the one read.	read → write	You read a forum post, then reply: every replica must apply your reply **after** the post you read.

Worked example: name the violated guarantee

Scenario	Violated
Alice sets her status to "busy" on her phone, then opens the app on her laptop and sees "available".	Read your writes
Bob's bank app shows balance 5,000 (after a deposit), refreshes and shows 3,000 (before the deposit).	Monotonic reads
Carol changes price to 100 then to 120; one replica ends with 100.	Monotonic writes
Dan reads a question and posts an answer; another user's replica shows the answer but not the question.	Writes follow reads

5. Replication (slides 18–22)

5.1 Why replicate data? (slide 19)

Benefit	How
Performance	Serve reads from a nearby replica: lower latency, higher throughput, load spread across servers
Reliability	If one replica fails or is partitioned away, others keep serving — no single point of failure
Scalability	Read-heavy workloads scale horizontally by adding replicas
Availability	Geographically distributed replicas keep the service reachable during regional outages

The catch: every benefit requires **keeping copies up to date**, which costs coordination, bandwidth and time. Replication and consistency are always in tension.

5.2 Update propagation: push vs pull (slide 20)



Figure 2 — Push vs pull update propagation.

	Push-based (server-initiated)	**Pull-based (client-initiated)**
How	Master pushes updates to replicas as soon as they occur	Replicas poll the server/cache for updates on demand
Freshness	Low read latency; replicas always (nearly) fresh	Reads may return stale data between polls
Bandwidth	Wasted if replicas are read infrequently	Saved: only fetch when actually needed
Good fit	**High read-to-write ratio**, need for strong freshness	**Low read-to-write ratio**, bandwidth-constrained clients

5.3 Placement of replicas (slide 21)

Type	**Who creates it**	**Where / example**
Permanent replicas	The fixed, initial set of servers that make up the data store	Core/origin: the primary data centres
Server-initiated replicas	Created dynamically **by the system** to improve performance	Regional: pushing a popular page's copy closer to a region with a traffic spike
Client-initiated replicas (caches)	Created at the **client's** request, usually temporary	Edge/client: a browser cache

Distance from clients increases from client caches (closest) → server-initiated → permanent (furthest, the core).

5.4 Consistency–replication trade-offs (slide 22)

Design choice	**Gains**	**Costs**
More replicas	Higher availability, read throughput	Harder to keep in sync, more write coordination
Stronger consistency	Simpler application logic, no stale reads	Higher write latency, lower availability under partition
Push propagation	Replicas stay fresh	Bandwidth cost even when unread
Pull propagation	Bandwidth-efficient	Readers may see stale data
Replicas placed near clients	Low read latency	More nodes to keep consistent; cache invalidation complexity

Every row trades consistency for availability/performance or vice versa — the bridge to CAP.

6. The CAP theorem and PACELC (slides 23–27)

Property	Formal definition (slide 24)
Consistency	Every read receives the most recent write **or an error**; all nodes see the same data at the same time.
Availability	Every request receives a **(non-error) response**, without the guarantee that it contains the most recent write.
Partition tolerance	The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network.

CAP theorem: a distributed data store can provide **at most two of these three** guarantees at the same time.

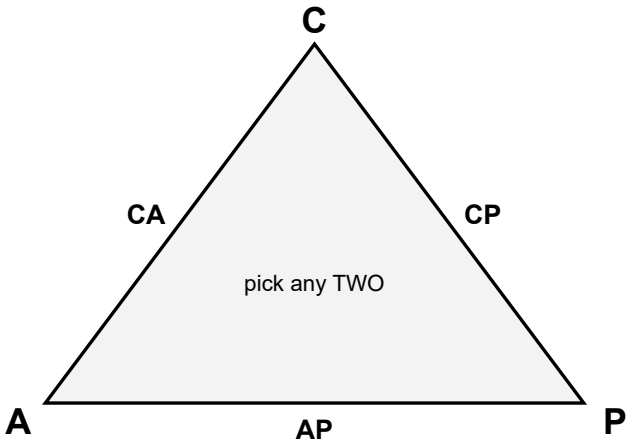


Figure 3 — The CAP triangle (slide 25): each edge is a pair.

- **CA**: consistent and available but cannot tolerate a partition — rare in practice; only works on a single site with no network split.
- **CP**: consistent and partition-tolerant — sacrifices availability: some nodes refuse requests during a split.
- **AP**: available and partition-tolerant — sacrifices strict consistency: nodes keep answering, possibly with stale data.

Summary sentence (slide 45): under a network partition you must choose Consistency or Availability — you always keep Partition tolerance.

Category	Behaviour under partition	Example-style systems (slide 26)
CP	Reject or queue requests on the minority side to keep data consistent	Distributed relational DBs with strong quorum writes; ZooKeeper-style coordination services
AP	Keep answering reads/writes on both sides; reconcile later	Highly available key-value/NoSQL stores; DNS; shopping-cart style services
CA	Only meaningful without a real network partition	A single-site database cluster on one reliable LAN

Note (slide 26): real products often let you **tune** the trade-off per operation (e.g. read/write **quorum** sizes) rather than being purely one category. **Quorum rule** (standard): with N replicas, a write must reach W replicas and a read must ask R replicas. If **R + W > N**, every read set overlaps every write set, so a read always sees the latest write (CP-like). Example: N = 3, W = 2, R = 2 ⇒ 4 > 3 ⇒ consistent; but if 2 replicas are unreachable, writes fail (less available). W = 1, R = 1 ⇒ 2 ≤ 3 ⇒ fast and available, but reads may be stale (AP-like).

PACELC — extending CAP (slide 27)

CAP only describes behaviour **during** a partition. But there is a trade-off even when the network is healthy: **if Partition (P), choose Availability (A) or Consistency (C); Else (E), choose Latency (L) or Consistency (C).** Why it matters: strong consistency needs coordination between replicas (waiting for acknowledgements), which adds latency even when nothing is broken.

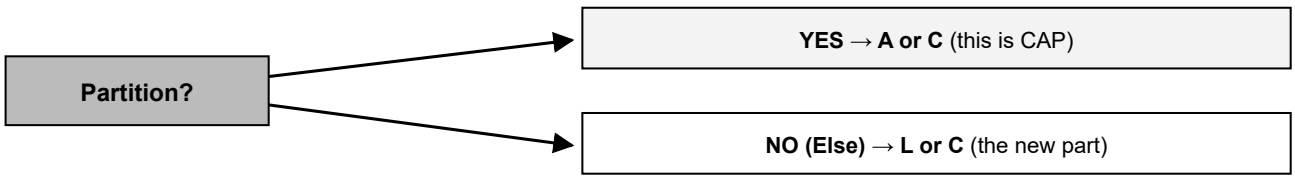


Figure 4 — PACELC decision path. Read as "if Partition → A or C; Else → L or C".

7. Fault tolerance (slides 28–32)

Fault tolerance: a system's ability to continue operating correctly despite the failure of some of its components.

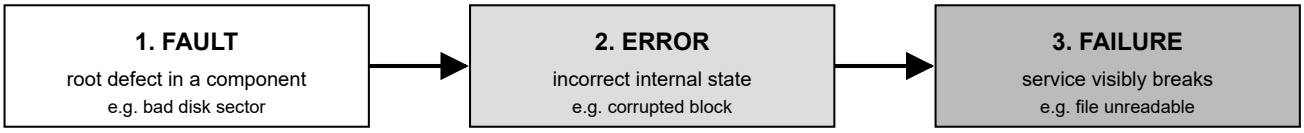


Figure 5 — Fault → error → failure (slide 29). Design goal: stop faults from becoming failures.

Design goal: prevent faults from becoming failures — **mask** them with redundancy, **detect** them quickly, and **recover** gracefully. **Dependability properties** (the pillars): **availability** (ready for use at any moment), **reliability** (runs continuously without failure over a period), **safety** (a failure causes nothing catastrophic), **maintainability** (easy to repair).

7.1 Types of failures (slide 30)

Failure type	Description	Severity
Crash	A process simply halts, but behaved correctly until it stopped	Low
Omission	A process fails to respond to incoming requests (send omission / receive omission)	Low–Medium
Timing	A response is correct but arrives outside a specified time interval	Medium
Response	The process responds incorrectly — wrong value or wrong control flow	Medium–High
Arbitrary (Byzantine)	Arbitrary output at arbitrary times, including malicious behaviour	Highest

Severity increases down the table: failures become harder to detect and tolerate.

7.2 Redundancy techniques (slide 31)

Technique	Idea	Examples
Information redundancy	Extra bits added to data so errors can be detected/corrected	Parity, checksums, error-correcting codes
Time redundancy	An action is performed, and if needed, repeated	Transaction retries, timeout + resend
Physical redundancy	Extra hardware or processes duplicate functionality	Replica servers, redundant network links, RAID disks

Replication (section 5) is physical redundancy applied specifically to data.

7.3 Process resilience: process groups (slide 32)

Idea: tolerate the failure of a process by organising several **identical** processes into a **group**; if one fails, others carry on.

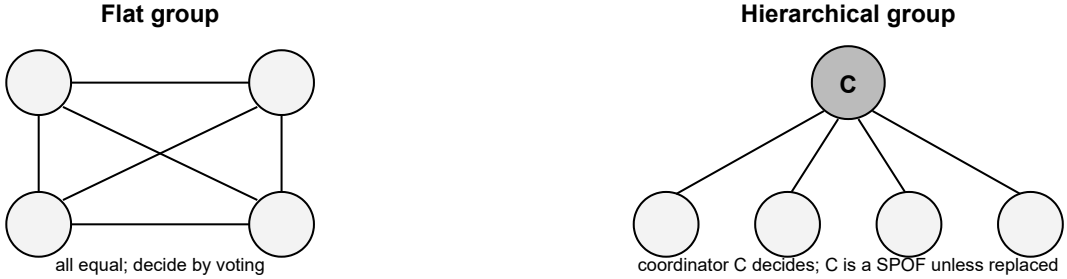


Figure 6 — Flat vs hierarchical groups (slide 32).

	Flat group	Hierarchical group
Structure	All processes equal/symmetric	One coordinator manages the workers
Strength	More robust: no single point of failure	Simpler, faster decisions
Weakness	Decisions need coordination (e.g. voting) ⇒ slower, more messages	Coordinator is a SPOF unless replaced automatically (an election picks a new one)

Both need a way to know when a member is gone: **failure detection**.

8. Failure detection: heartbeat and gossip (slides 33–35)

8.1 Heartbeat protocol

Each process periodically sends an **"I'm alive" (heartbeat)** message to a monitor (or its neighbours). **Detection rule:** if no heartbeat arrives within a **timeout** window, the process is **suspected** failed. Simple, low overhead; but one missed heartbeat caused by network delay (not a crash) can give a **false positive**. **Tuning trade-off:** shorter intervals detect failures faster but create more traffic and more false suspicions.

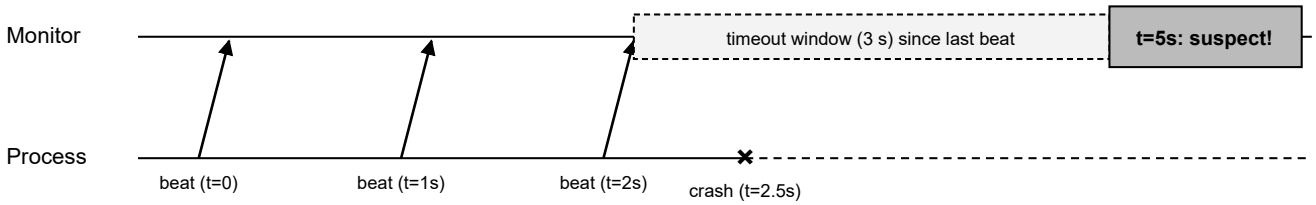


Figure 7 — Heartbeat every 1 s, timeout 3 s. Last beat at t = 2 s, so the monitor suspects failure at t = 5 s (detection delay 2.5 s after the crash).

8.2 Gossip protocol (decentralized)

Each process periodically exchanges membership/state information with a few **randomly chosen peers**, not a central monitor. Information spreads **epidemically**, like a rumour: updates (including failure suspicions) reach the whole system in **O(log N)** rounds. Highly scalable and resilient: no single point of failure, works in very large, dynamic clusters. Trade-off: detection is **probabilistic** and takes a few rounds to spread — not instantaneous.

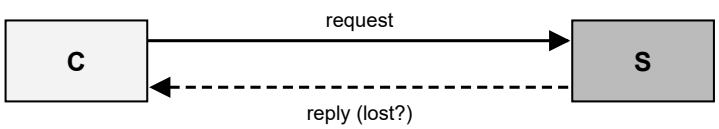
Worked example (why log N): if every informed node tells one new random node each round, the number informed roughly doubles: 1 → 2 → 4 → 8 … After r rounds about 2^r nodes know. For N = 1,000 nodes: 2^{10} = 1,024 ⇒ about **10 rounds**; for N = 1,000,000 only about 20 rounds. (Real gossip needs a few extra rounds because random picks sometimes hit already-informed nodes.)

Aspect (slide 35)	Heartbeat	Gossip
Architecture	Centralized (monitor watches processes)	Decentralized (peer-to-peer exchange)
Scalability	Limited — monitor can bottleneck	High — load spread across all nodes
Detection speed	Fast, bounded by timeout	Slower, probabilistic, spreads over rounds
Single point of failure	Yes (the monitor)	No
Best fit	Small-to-medium clusters, simple setups	Very large, dynamic, peer-to-peer systems

9. Reliable communication (slides 36–39)

9.1 Reliable client-server (point-to-point)

Point-to-point reliability usually builds on a reliable transport like **TCP**, but **reliable bytes are not the same as a reliable RPC** (Remote Procedure Call: calling a function on another machine). Things that can go wrong:



1. Request lost in transit
2. Server crashes mid-execution
3. Reply lost in transit
4. Client crashes awaiting reply

Figure 8 — Where client-server communication can go wrong (slide 37).

- **Failure at the client**: the client crashes before getting the reply ⇒ the server's work becomes an **orphan computation** that needs cleanup.
- **Failure at the server**: did it crash **before or after** executing the request? The client often cannot tell. If it retries, the operation might run twice.

Semantics	Guarantee	How / when it fits
At-most-once	The operation runs 0 or 1 times, never twice	Do not retry blindly (or detect duplicate requests). Good for non-repeatable actions like payments, but the action may not happen at all.
At-least-once	Runs 1 or more times	Keep retrying until a reply arrives. Safe only for **idempotent** operations (same result if repeated, e.g. "set balance = 500", "read file").
Exactly-once	Runs exactly one time	Ideal but hard: needs request IDs, duplicate filtering and stored results, plus recovery.

Trade-off on the slide: simplicity vs correctness. Example: "withdraw Rs. 1000" with at-least-once could withdraw twice if the reply was lost and the client retried.

9.2 Reliable group (one-to-many) communication

Goal: reliably deliver one message to **all** members of a process group, even if some members or the sender fail mid-multicast. **Multicast** = one sender, many receivers.

- **Atomic multicast**: a message is delivered to **every non-faulty member or to none** — no partial delivery.
- **Challenge**: the sender may crash partway through sending; the group must agree whether the message "counts" as delivered.
- Slide 39 demo: the sender multicasts to five members; one (×) misses it. Atomic multicast either **resends** or **rolls back** so all non-faulty members end up consistent.

Ordering guarantee	Meaning	Example
FIFO order	Messages from **one sender** arrive in the order sent	Alice's m1 then m2 is delivered as m1, m2 everywhere
Causal order	Causally related messages, from any senders, respect happened-before	A reply is delivered after the message it answers
Total order	**All** members see **all** messages in the **same** order	Every replica applies updates u1, u2, u3 in one agreed order

10. Distributed commit protocols (slides 40–43)

A **distributed transaction** changes data on several machines (e.g. a transfer debits a Commercial Bank account and credits an HNB account). It must be **atomic**: ALL parts commit or ALL abort — no partial commits.

10.1 Two-Phase Commit (2PC)

Roles: one **coordinator** drives the protocol; several **participants** (cohorts) hold pieces of the transaction.

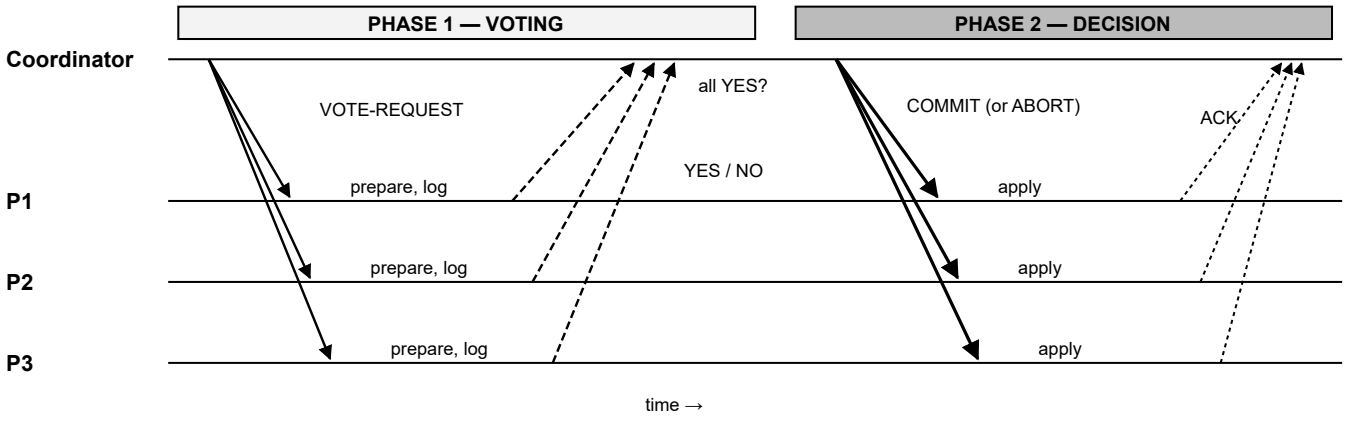


Figure 9 — 2PC step by step (slide 41). If ANY participant votes NO, the coordinator broadcasts ABORT and everyone rolls back.

1. **Phase 1 — Voting:** the coordinator sends VOTE-REQUEST ("can you commit?"). Each participant checks it can (locks held, constraints OK), writes its work to a durable log, and votes **YES (ready)** or **NO (abort)**. After voting YES, a participant has promised and may not change its mind.
2. **Phase 2 — Decision:** if **all** voted YES, the coordinator broadcasts **COMMIT**; if **any** voted NO (or did not answer in time), it broadcasts **ABORT**. Participants apply the decision and release locks.

Worked example: 2PC traces

Case	What happens	Outcome
P1 YES, P2 YES, P3 YES	Coordinator sends COMMIT; all apply	Committed everywhere
P1 YES, P2 NO, P3 YES	One NO ⇒ coordinator sends ABORT; P1 and P3 roll back	Aborted everywhere (atomicity kept)
P3 crashes before voting	Coordinator times out waiting ⇒ treats as NO ⇒ ABORT	Aborted
All vote YES, then the coordinator crashes before sending the decision	Participants are in the "ready" state: they cannot COMMIT (maybe someone voted NO) and cannot ABORT (maybe the coordinator decided COMMIT and some already applied it). They must **wait, holding locks**, until the coordinator recovers.	**BLOCKED**

KEY WEAKNESS OF 2PC (SLIDE 40)

2PC is a **blocking protocol**: if the coordinator crashes after participants vote YES but before they hear the decision, they are stuck — they cannot safely commit or abort on their own and must wait (potentially holding locks) until the coordinator recovers. This is what 3PC is designed to fix.

10.2 Three-Phase Commit (3PC)

Motivation: remove 2PC's blocking problem by adding a buffer phase before the final commit.

1. **Phase 1 — Voting:** identical to 2PC: VOTE-REQUEST → YES/NO.
2. **Phase 2 — Prepare-to-commit:** if all voted YES, the coordinator sends **PREPARE-TO-COMMIT** (not a final commit yet). Participants acknowledge (**ACK**) — they now know a commit **is** coming and that everyone voted YES.
3. **Phase 3 — Commit:** the coordinator sends the final **COMMIT**; participants apply it.

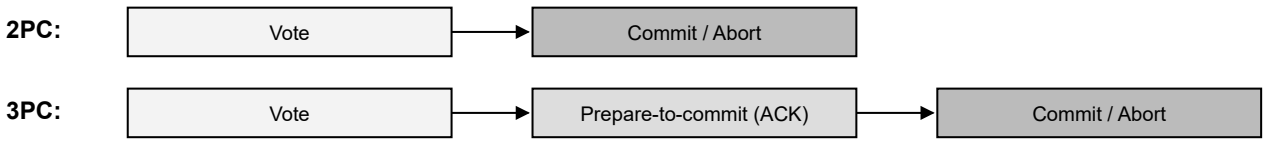


Figure 10 — 2PC vs 3PC phases (slide 42).

Why it doesn't block: the extra middle step means every participant knows the group's **intent** before anyone commits. With timeouts, a recovering or newly elected coordinator can safely infer the outcome:

- If **any** surviving participant has received PREPARE-TO-COMMIT, then everyone voted YES ⇒ it is safe to **COMMIT**.
- If **no** survivor has received it, no one can have committed yet ⇒ it is safe to **ABORT**.

Slide 43: if the coordinator fails after Phase 2, surviving participants can elect a new coordinator and COMMIT, because they already know everyone was ready. This holds **under crash-only (non-Byzantine) failures and no network partition**; with a partition, 3PC can still go wrong.

2PC	3PC
2 phases	3 phases
Can block on coordinator crash	Avoids blocking (crash-only faults)
Lower message overhead	Higher message overhead (extra round)
Simpler to implement	More complex, extra timeouts

11. Security and access control in distributed file systems (slide 44)

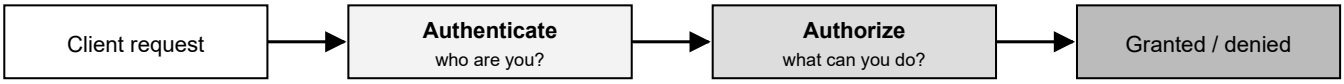


Figure 11 — Access control flow (slide 44).

Mechanism	Meaning
Authentication	Verifying **who** is making a request: Kerberos-style **ticket** systems (log in once to a trusted server, get a time-limited ticket to show to file servers), **mutual TLS** certificates (both client and server prove identity).
Authorization / access control	Deciding **what** an authenticated user may do: **ACLs** (access control lists: each file stores a list of users and their rights), **capability-based** access (the user holds an unforgeable token naming the object and rights), **RBAC** (role-based: rights are given to roles like "lecturer", users get roles).
Integrity & confidentiality	Encryption **in transit** (TLS) and **at rest** (on disk), plus checksums/signatures to detect tampering.
Replication risk	Every replica is another copy an attacker could target; the access-control policy must be enforced **consistently across all replicas**.
Auditing	Logging access and changes so breaches or misuse can be detected and traced afterwards.

ACL vs capability: an ACL is stored *with the object* ("file F: Alice read/write, Bob read"); a capability is held *by the subject* ("Bob's token: read F"). ACLs make it easy to see/revoke who can access a file; capabilities make it easy to delegate and check quickly without a central lookup.

12. Summary (slide 45)

- **Consistency:** data-centric models define global rules; client-centric models protect one client's personal view.
- **Replication:** push/pull propagation and replica placement (permanent, server-, client-initiated) trade freshness for cost.
- **CAP:** under a partition, choose C or A; always keep P. PACELC adds latency vs consistency when there is no partition.
- **Fault tolerance:** faults → errors → failures; redundancy (information, time, physical) masks faults before they become failures.
- **Failure detection:** heartbeat = centralized and fast; gossip = decentralized and scalable.
- **Commit protocols:** 2PC is simple but can block; 3PC adds a phase to avoid blocking under crash-only failures.
- **Security:** authentication + authorization + encryption, enforced consistently on every replica.

13. Exam-style questions with model answers

Q1. What is a consistency model? Distinguish data-centric and client-centric models with one example of each. (6 marks)
A consistency model is a contract between a data store and processes that specifies what values a read may return after a sequence of writes. Data-centric models define consistency from the data store's view, as rules on the ordering of operations by all processes on shared data (e.g. sequential consistency: all processes see the same interleaving). Client-centric models define it from one (mobile) client's view across successive accesses to different replicas (e.g. read-your-writes: a process always sees its own earlier writes).

Q2. Compare strict, sequential, causal and eventual consistency. (8)
Strict: every read returns the most recent write in real time, as with one global clock and one copy — strongest, not achievable in practice. Sequential: all processes see the same single interleaving of all operations, respecting each process's program order but not real time — implementable. Causal: only causally related writes must be seen in the same order by everyone; concurrent writes may be seen in different orders — cheaper. Eventual: if updates stop, all replicas eventually converge; no ordering or timing guarantee in between — weakest, most available (DNS, NoSQL). Performance and availability increase as guarantees weaken.

Q3. P1: W(x)a. P2: W(x)b. P3: R(x)b, R(x)a. P4: R(x)a, R(x)b. Is this sequentially consistent? Explain. (4)
No. P3 observes b before a, while P4 observes a before b. Sequential consistency requires all processes to see the same interleaving of writes, and no single order of W(x)a and W(x)b explains both P3's and P4's reads.

Q4. Explain the four client-centric consistency guarantees with an example each. (8)
Monotonic reads: once a process has read a value it never later reads an older one (a phone must not show fewer emails than the laptop already showed). Monotonic writes: a process's writes are applied on every replica in the order issued (title update before body update). Read your writes: a process always sees its own writes (a posted comment appears after refresh). Writes follow reads: a write is applied on the same or newer state than the value previously read (a reply is ordered after the post it answers).

Q5. Compare push-based and pull-based update propagation. When is each suitable? (5)
Push: the server sends updates to replicas immediately; replicas stay fresh and reads are fast, but bandwidth is wasted if replicas are rarely read; suitable for a high read-to-write ratio and strong freshness needs. Pull: replicas poll for updates when needed; bandwidth-efficient but reads may be stale between polls; suitable for a low read-to-write ratio and bandwidth-limited clients.

Q6. Describe the three types of replica placement. (3)
Permanent replicas: the fixed initial set of servers of the data store (origin data centres). Server-initiated replicas: created dynamically by the system to handle demand (copy of a popular page pushed to a busy region). Client-initiated replicas: caches created at the client's request, usually temporary (browser cache).

Q7. State the CAP theorem. Classify ZooKeeper, DNS and a single-site database cluster. What does PACELC add? (7)
A distributed data store can provide at most two of consistency, availability and partition tolerance. ZooKeeper-style coordination services are CP (reject requests on the minority side to stay consistent). DNS is AP (keeps answering, possibly stale, converges later). A single-site cluster on one reliable LAN is CA (meaningful only without partitions). PACELC adds that when there is no partition (Else), there is still a trade-off between Latency and Consistency, because strong consistency requires coordination that adds latency.

Q8. Explain fault, error and failure with an example. Name the three redundancy techniques. (6)
A fault is a defect in a component (a bad disk sector); it can cause an error, an incorrect internal state (a corrupted block); if the error reaches the service, a failure occurs, a visible breakdown (file unreadable). Redundancy: information (parity, checksums, ECC), time (retry, timeout and resend), physical (replica servers, redundant links, RAID).

Q9. Compare heartbeat and gossip failure detection. (6)
Heartbeat: each process periodically sends "I'm alive" to a monitor; no heartbeat within the timeout means suspected failure. Centralized, fast and bounded detection, simple, but the monitor is a bottleneck and single point of failure, and delays cause false positives. Gossip: processes exchange membership state with random peers; information spreads epidemically in O(log N) rounds. Decentralized, highly scalable, no single point of failure, but detection is probabilistic and slower. Heartbeat suits small/medium clusters; gossip suits very large dynamic systems.

Q10. Explain at-most-once, at-least-once and exactly-once semantics. Why is a lost reply a problem? (6)
At-most-once: the request is executed zero or one times (no duplicates, but may not run). At-least-once: retried until a reply arrives, so it runs one or more times; safe only for idempotent operations. Exactly-once: runs exactly once; ideal but needs duplicate detection and recovery. A lost reply is a problem because the client cannot tell whether the server executed the request; retrying may execute it twice, not retrying may leave it undone.

Q11. What is atomic multicast? Explain FIFO, causal and total ordering. (6)
Atomic multicast delivers a message to every non-faulty group member or to none, even if the sender crashes midway. FIFO: messages from one sender arrive in the order sent. Causal: causally related messages from any senders are delivered respecting happened-before. Total: all members deliver all messages in the same order.

Q12. Describe Two-Phase Commit. Explain its blocking problem and how Three-Phase Commit solves it. (10)
2PC has a coordinator and participants. Phase 1 (voting): the coordinator sends VOTE-REQUEST; each participant votes YES (ready) or NO. Phase 2 (decision): if all vote YES the coordinator broadcasts COMMIT; if any votes NO it broadcasts ABORT; all participants follow, giving all-or-nothing atomicity. Blocking: if the coordinator crashes after participants voted YES but before they receive the decision, they cannot commit or abort on their own and must wait, holding locks, until it recovers. 3PC adds a prepare-to-commit phase: after all YES votes the coordinator sends PREPARE-TO-COMMIT and waits for ACKs, then sends COMMIT. Every participant therefore knows the group's intent before anyone commits, so after a coordinator crash the survivors (or a new coordinator) can use timeouts: if

any has received PREPARE-TO-COMMIT they commit, otherwise they abort. This avoids indefinite blocking under crash-only failures without partitions, at the cost of an extra round of messages and more complexity.

Q13. Identify the core access-control mechanisms used to secure a distributed file system. (6)
Authentication (who: Kerberos tickets, mutual TLS); authorization (what: ACLs, capabilities, RBAC); integrity and confidentiality (TLS in transit, encryption at rest, checksums/signatures); consistent policy enforcement on every replica; auditing (logging access and changes).

14. Common mistakes

Mistake	Correct idea
Sequential consistency must follow real time	Only strict consistency needs real time; sequential needs one agreed interleaving.
Causal consistency orders all writes	Only causally related writes; concurrent writes may be seen in different orders.
Eventual consistency = data may be lost	All replicas converge to the last value once updates stop; reads may only be temporarily stale.
Mixing up monotonic reads and read-your-writes	Monotonic reads: read → read never goes back. Read-your-writes: write → read sees your own write.
Push is better for rarely-read replicas	Push wastes bandwidth there; pull fits a low read-to-write ratio.
CAP availability = 99.99% uptime	CAP availability = every request gets a non-error response.
Fault and failure used as synonyms	Fault (cause) → error (bad state) → failure (visible).
Heartbeat is decentralized	Heartbeat uses a monitor (centralized); gossip is decentralized.
2PC blocks when a participant votes NO	A NO just causes ABORT. Blocking happens when the **coordinator crashes** after YES votes, before the decision.
3PC works under any failure	Only crash-only failures without network partitions.
Retrying any request is safe	Only idempotent ones; otherwise at-least-once can execute twice.

Cheat sheet

Consistency	
Model	Contract: what a read may return after writes
Strict	Latest write in real time; needs global clock; not achievable
Sequential	Same interleaving for all; own program order; not real time
Causal	Related writes same order; concurrent any order
Eventual	No new updates ⇒ converge; DNS, NoSQL
Mono. reads	read→read: never older
Mono. writes	write→write: your order everywhere
Read your writes	write→read: see own write
Writes follow reads	read→write: write on same/newer state

Replication	
Why	Performance, reliability, scalability, availability
Push	Server-initiated; fresh; high read:write
Pull	Client-initiated; saves bandwidth; low read:write
Placement	Permanent (core) · server-initiated (regional) · client-initiated caches (edge)
Quorum	R + W > N ⇒ reads see latest write

CAP / PACELC	
CP	Refuse minority side: quorum DBs, ZooKeeper
AP	Answer both sides, reconcile: NoSQL, DNS, carts
CA	Single site, one LAN
PACELC	P: A or C; Else: L or C

Fault tolerance	
Chain	Fault → error → failure
Dependability	Availability, reliability, safety, maintainability
Failures	Crash < omission < timing < response < Byzantine
Redundancy	Information (parity, ECC) · time (retry) · physical (replicas, RAID)
Groups	Flat (robust, voting) · hierarchical (simple, coordinator SPOF)
Heartbeat	"I'm alive" to monitor; timeout ⇒ suspect; fast; SPOF; false positives
Gossip	Random peers; epidemic; O(log N) rounds; scalable; probabilistic

Reliable communication	
C-S failures	Request lost, server crash, reply lost, client crash (orphans)
Semantics	At-most-once · at-least-once (idempotent) · exactly-once
Atomic multicast	All non-faulty members or none
Ordering	FIFO (per sender) · causal · total (same order for all)

Commit & security	
2PC	VOTE-REQUEST → YES/NO → COMMIT (all yes) / ABORT (any no). **Blocks** if coordinator dies after YES votes
3PC	Vote → PREPARE-TO-COMMIT + ACK → COMMIT. Non-blocking (crash-only, no partitions); more messages
DFS security	Authenticate (Kerberos, mTLS) → authorize (ACL, capability, RBAC) → encrypt (TLS, at rest) + checksums; same policy on all replicas; auditing